Stochastic Gradient Descent (SGD) Optimizer

SGD class

```python
class SGD:
    def __init__(self, lr=0.01, epochs=1000, batch_size=32, tol=1e-3):
        self.learning_rate = lr
        self.epochs = epochs
        self.batch_size = batch_size
        self.tolerance = tol
        self.weights = None
        self.bias = None

    def predict(self, X):
        return np.dot(X, self.weights) + self.bias

    def mean_squared_error(self, y_true, y_pred):
        return np.mean((y_true - y_pred) ** 2)

    def gradient(self, X_batch, y_batch):
        y_pred = self.predict(X_batch)
        error = y_pred - y_batch
        gradient_weights = np.dot(X_batch.T, error) / X_batch.shape[0]
        gradient_bias = np.mean(error)
        return gradient_weights, gradient_bias

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.random.randn(n_features)
        self.bias = np.random.randn()

        for epoch in range(self.epochs):
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            for i in range(0, n_samples, self.batch_size):
                X_batch = X_shuffled[i:i+self.batch_size]
                y_batch = y_shuffled[i:i+self.batch_size]

                gradient_weights, gradient_bias = self.gradient(X_batch, y_batch)
                self.weights -= self.learning_rate * gradient_weights
                self.bias -= self.learning_rate * gradient_bias

            if epoch % 100 == 0:
                y_pred = self.predict(X)
                loss = self.mean_squared_error(y, y_pred)
                print(f"Epoch {epoch}: Loss {loss}")

            if np.linalg.norm(gradient_weights) < self.tolerance:
                print("Convergence reached.")
                break

        return self.weights, self.bias
```

SGD Implementation

```
# Create random dataset with 100 rows and 5 columns
X = np.random.randn(100, 5)
# create corresponding target value by adding random
# noise in the dataset
y = np.dot(X, np.array([1, 2, 3, 4, 5]))\
    + np.random.randn(100) * 0.1
# Create an instance of the SGD class
model = SGD(lr=0.01, epochs=1000,
            batch_size=32, tol=1e-3)
w,b=model.fit(X,y)
# Predict using predict method from model
y_pred = w*X+b
#y_pred
```

Binary Classification using SGD

```
import os
import zipfile
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from sklearn.model_selection import train_test_split

# Download and extract the dataset
dataset_url = "https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip"
dataset_path = "cats_and_dogs_filtered.zip"

# Download the dataset if not already downloaded
if not os.path.exists(dataset_path):
    print("Downloading dataset...")
    import requests
    response = requests.get(dataset_url)
    with open(dataset_path, "wb") as file:
        file.write(response.content)

# Extract the dataset
if not os.path.exists("cats_and_dogs_filtered"):
    print("Extracting dataset...")
    with zipfile.ZipFile(dataset_path, "r") as zip_ref:
        zip_ref.extractall()

# Set paths
base_dir = "cats_and_dogs_filtered"
train_dir = os.path.join(base_dir, "train")
validation_dir = os.path.join(base_dir, "validation")
```

```
# Prepare image data and labels
def load_images_and_labels(directory, label, image_size=(64, 64)):
    images = []
    labels = []
    for filename in os.listdir(directory):
        file_path = os.path.join(directory, filename)
        try:
            img = Image.open(file_path).convert("L")  # Convert to grayscale
            img = img.resize(image_size)
            images.append(np.array(img))
            labels.append(label)
        except:
            continue
    return np.array(images), np.array(labels)

# Load training data
cats_train_dir = os.path.join(train_dir, "cats")
dogs_train_dir = os.path.join(train_dir, "dogs")
X_cats, y_cats = load_images_and_labels(cats_train_dir, label=0)
X_dogs, y_dogs = load_images_and_labels(dogs_train_dir, label=1)

# Combine cats and dogs data
X_train = np.concatenate([X_cats, X_dogs], axis=0)
y_train = np.concatenate([y_cats, y_dogs], axis=0)

# Normalize the data
X_train = X_train / 255.0

# Flatten the images
X_train_flat = X_train.reshape(X_train.shape[0], -1)
```

```
# Split the data into training and validation sets
X_train_flat, X_val_flat, y_train, y_val = train_test_split(X_train_flat,
y_train, test_size=0.2, random_state=42)

# Model parameters
input_size = X_train_flat.shape[1]
W = np.random.randn(input_size) * 0.01  # Initialize weights with small
random values
b = 0  # Initialize bias
learning_rate = 0.01  # Learning rate
epochs = 10  # Number of epochs
batch_size = 32  # Batch size

# List to store the training loss for each epoch
losses = []

# Define sigmoid and binary cross-entropy functions
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

```
def binary_cross_entropy(y_true, y_pred):
    # Avoid log(0) by clipping predictions
    y_pred = np.clip(y_pred, 1e-8, 1 - 1e-8)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 -
y_pred))

# Training loop
for epoch in range(epochs):
    # Shuffle the training data at the start of each epoch
    indices = np.arange(X_train_flat.shape[0])
    np.random.shuffle(indices)
    X_train_flat = X_train_flat[indices]
    y_train = y_train[indices]

    for i in range(0, X_train_flat.shape[0], batch_size):
        # Mini-batch gradient descent
        X_batch = X_train_flat[i:i + batch_size]
        y_batch = y_train[i:i + batch_size]

        # Forward pass
        y_pred = sigmoid(np.dot(X_batch, W) + b)

        # Compute gradients
        error = y_pred - y_batch
        dW = np.dot(X_batch.T, error) / batch_size
        db = np.sum(error) / batch_size

        # Update weights and bias
        W -= learning_rate * dW
        b -= learning_rate * db

    # Calculate loss for the entire training set
    y_pred_train = sigmoid(np.dot(X_train_flat, W) + b)
    loss = binary_cross_entropy(y_train, y_pred_train)
    losses.append(loss)
    print(f"Epoch {epoch + 1}/{epochs}, Loss: {loss:.4f}")
```

```
# Plot the training loss over epochs
plt.plot(losses, label="SGD Loss")
plt.title("Training Loss with SGD Optimizer")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Evaluate the model on the validation set
y_pred_val = sigmoid(np.dot(X_val_flat, W) + b)
y_pred_val_binary = (y_pred_val >= 0.5).astype(int)
accuracy = np.mean(y_pred_val_binary == y_val)
print(f"Validation Accuracy with SGD: {accuracy:.4f}")
```

Assignment

- Implement and compare the performance of the **Mini Batch SGD**, **ADAGRAD**, **RMSProp**, **ADAM**, and **SGD with Momentum** optimizers for training a binary classification model using the Cats and Dogs dataset. Preprocess the dataset by converting images to grayscale, resizing them to 64x64, normalizing pixel values to [0, 1], and splitting the data into training and validation sets. Train the model for 10 epochs using each optimizer, visualize the training loss by plotting it, and compute the validation accuracy.

Thank you